DynaFed Whitepaper

Fabio Lama
BotanixLabs
fabio@botanixlabs.xyz

Darius Parvin
BotanixLabs
darius@botanixlabs.xyz

November 27, 2025
Revision 1 / Proof-of-Concept

Edited & refined by:
Claude.ai (Anthropic)

© 2025 BotanixLabs
CC BY 4.0

Abstract

DynaFed is a dynamic federation management protocol designed to enable secure and efficient Bitcoin custody for the Botanix layer 2 network. The protocol addresses the fundamental tension between security and operational efficiency in FROST threshold signature schemes by introducing a two-tier architecture: Tier 1 pools serve as large, conservative cold storage federations holding the majority of bridged funds, while Tier 2 pools operate as smaller, more flexible hot wallets optimized for low-latency withdrawal fulfillment. FROST's multi-round coordination protocol and static membership requirements create scalability bottlenecks that would be prohibitive for a single large federation, necessitating this architectural separation. DynaFed dynamically directs Bitcoin deposits and withdrawals based on real-time collateralization levels and network demand, using adaptive capacity estimation to adjust hot wallet reserves to changing withdrawal patterns. The system leverages Bitcoin's transaction output model as a core safeguard for operation atomicity and employs a lightweight fork-awareness algorithm that enables deterministic finality detection without external oracles. By maintaining collateralized hot wallets backed by cold storage reserves, DynaFed achieves a balance between the security guarantees of large federations and the operational responsiveness required for a production Bitcoin layer 2 network.

Contents

1 Introduction **3**
- 1.1 Document Structure 3

2 Architecture **5**
- 2.1 Block Proposal Construction 5
- 2.2 Block Proposal Validation & Finalization 6
- 2.3 System Flow: Pegin Lifecycle 6
- 2.4 System Flow: Pegout Lifecycle 6

3 The FROST Challenge **6**

4 Two-Tier Federations **7**
- 4.1 Tier 1 Architecture 7
- 4.2 Tier 2 Architecture 7
  - 4.2.1 Future Extensibility 8
- 4.3 Staking, Deposits & Collateralization . . 8

5 Pool Management **8**
- 5.1 Dynamic Capacity Target 8
- 5.2 Collateralization Level 9
- 5.3 Pegin Allocation Logic 9
- 5.4 Pegout Allocation Logic 9
  - 5.4.1 Future Enhancements 10

6 Bitcoin Fork Awareness **10**
- 6.1 Block Tree Integration 10

7 Risk Assessment **10**
- 7.1 Risk Level 11
  - 7.1.1 State Control Parameter 11
  - 7.1.2 Emergency Controller 12
  - 7.1.3 Participation Score 12
  - 7.1.4 Tenure Score 13

8 The Foundation Layer **13**
- 8.1 Pegout Proposals 14
  - 8.1.1 Updating Proposal 14
  - 8.1.2 Cancelling Proposal 14
- 8.2 Foundation State Proof 15
  - 8.2.1 State Proof Structure 15
  - 8.2.2 Context Metadata 15
  - 8.2.3 Auxiliary Events 15

9 Appendix **16**
- 9.1 DKG: Message Complexity 16
  - 9.1.1 Comparison 16
  - 9.1.2 With Bundling 16
  - 9.1.3 Without Bundling 16
- 9.2 Block Tree 17
  - 9.2.1 Core Definitions 17
  - 9.2.2 Block Insertion and Pruning . . 17
  - 9.2.3 Pruning Criteria 18
  - 9.2.4 Backward Pruning (Orphaned) . 18
  - 9.2.5 Forward Pruning (Finalized) . . 18
  - 9.2.6 Block Classification 18

1 Introduction

Botanix is an EVM-compatible layer 2 blockchain built on the Bitcoin network. Unlike most layer 2 systems that introduce new cryptocurrencies, Botanix employs a 1-to-1 peg with Bitcoin: users mint tokens by constructing a **pegin** through a verified Bitcoin transaction, and withdraw funds by constructing a **pegout** that burns tokens on Botanix and triggers a corresponding Bitcoin transaction.

Bitcoin's limited state introspection capabilities make it very hard to implement smart contracts that hold Bitcoin on behalf of external networks or validate their state transitions natively. This fundamental constraint drives the complexity addressed in this work: designing a secure, permissionless multisig system using FROST threshold signatures that can custody Bitcoin funds while maintaining the security and operational requirements of a production layer 2 network. The coordination complexity inherent to FROST, particularly its multi-round signing protocol and static membership requirements, fundamentally shapes our architectural decisions.

Botanix operates through three coordinated state layers, each producing cryptographic commitments that validators use to verify system-wide consistency:

- **CometBFT**: Provides Byzantine Fault Tolerant consensus for block producer selection and transaction ordering
- **Application Layer**: Executes transactions and maintains deterministic EVM state, producing an EVM state commitment
- **Foundation Layer**: Coordinates multisig federation membership, pegout lifecycle management, and Bitcoin fork awareness, producing a Foundation state commitment

The Foundation Layer, which is the primary focus of this paper, addresses two key challenges: managing federation rotations despite FROST's static membership requirements, and tracking pegouts across Bitcoin's probabilistic finality model where chain reorganizations can "undo" previously confirmed transactions. These constraints motivate the two-tier federation architecture that balances security and operational flexibility.

1.1 Document Structure

This paper is organized into the following sections:

1. **FROST Challenges & Two-Tier Architecture**: Section 3 describes the challenges inherent to the FROST protocol and motivates our two-tier federation system. Section 4 presents the architecture and analyzes the security-efficiency tradeoffs.
2. **Pool Management**: Section 5 presents the theory behind pegin and pegout allocation, defining how we balance the two-tier federation system securely and reliably.
   - **Dynamic capacity targeting** (section 5.1): Maintains sufficient liquid funds in Tier 2 federations by adapting to pegout demand patterns
   - **Real-time collateralization** (section 5.2): Ensures fund safety by tracking collateral backing for Tier 2 federations, mitigating risks from their smaller size
3. **Risk Assessment & Lifecycle**: Section 7 describes how DynaFed determines when new Tier 1 or Tier 2 federations must be established and how funds should be safely rotated to newly established federations.
   - We define a reproducible risk metric that governs rotation urgency, favoring a conservative approach unless degradation is detected
4. **Pegout Proposals & Bitcoin Finality**: Section 8.1 addresses the challenges of tracking pegout proposals once federation members construct Bitcoin L1 transactions.
   - We examine how "executed" pegouts should be tracked across Bitcoin's probabilistic finality model and identify which assumptions cannot be made
   - We leverage the UTXO-reuse property to prevent double-execution of pegouts in cases of orphaned Bitcoin blocks (where transactions return to mempool), proposal upgrades/fee bumps (section8.1.1), or cancellations (section 8.1.2)
   - We present a lightweight Bitcoin fork awareness mechanism (section 6) that tracks pegout proposals across competing forks and orphaned blocks, enabling deterministic finality detection despite Bitcoin's probabilistic consensus
5. **Foundation Layer Integration**: Section 8 describes the main component that coordinates all subsystems. The Foundation Layer interprets Botanix-specific consensus messages and orchestrates:
   - Pegin and pegout routing decisions
   - Dynamic capacity target and real-time collateralization calculations
   - Risk metric computation and federation lifecycle management (construction, rotation, sunset)
   - Bitcoin block and pegout proposal tracking with fork awareness

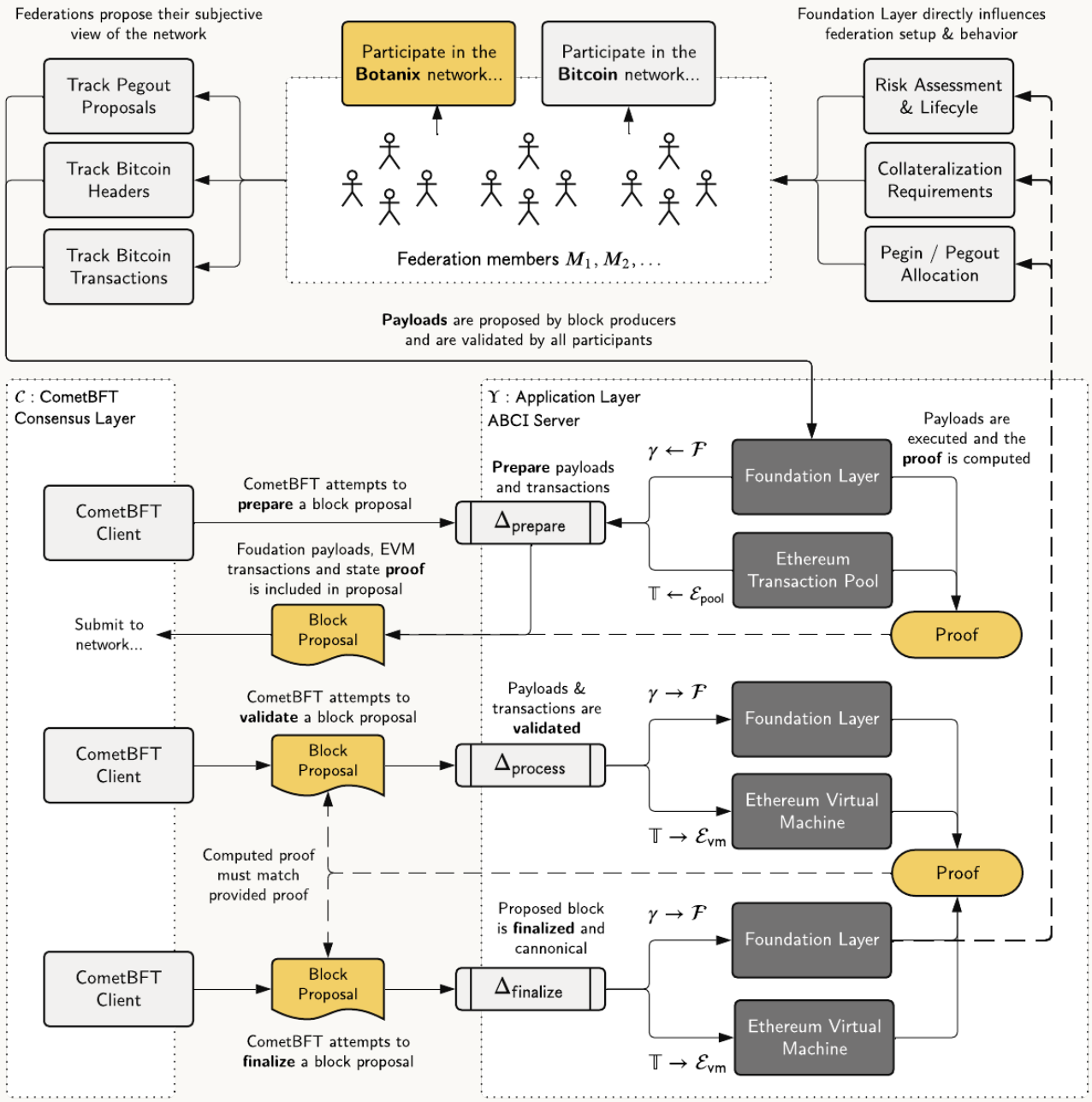
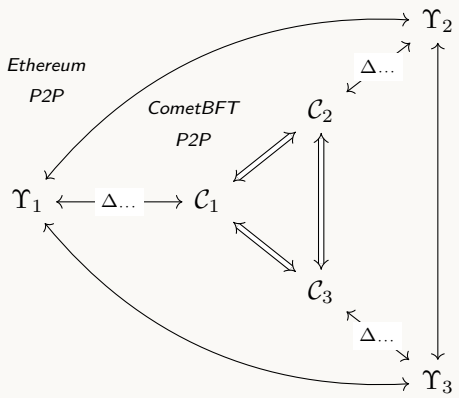


Figure 1: DynaFed System Architecture

2 Architecture

The consensus engine is the core component of any permissionless blockchain system. Botanix uses CometBFT, one of the most adopted and robust Proof-of-Stake (PoS) engines, which acts as middleware between arbitrary application logic and the consensus layer. While CometBFT follows its own specification, it provides sufficient flexibility for us to control transaction structure, state transition interpretation, and validator selection decisions. CometBFT treats application logic as a black box; from its perspective, both transactions and their resulting state transitions are opaque.

CometBFT operates as a standalone client $\mathcal{C}$ that communicates directly with other CometBFT clients, handling all consensus networking internally. However, because it is application-agnostic, CometBFT delegates transaction processing and state management to the application layer through the ABCI interface Δ. The application logic Υ implements a handful of ABCI methods that the client uses to prepare block proposals, validate proposals, and finalize blocks.



Three validators communicating, where each pair $(\mathcal{C}_n, \Upsilon_n)$ represents a complete validator node. $\mathcal{C}_n$ is the CometBFT consensus client and Υ_n is the application layer.

Since Botanix is an EVM-compatible chain, we reuse Ethereum's execution layer while replacing Ethereum's native consensus with CometBFT. Currently, all network participants (validators and non-validators alike) must run the CometBFT client, which handles transaction gossiping and block finalization. Ethereum's P2P block propagation is disabled in favor of CometBFT's networking layer. Future protocol versions may revisit this architecture to allow non-validators to sync via alternative mechanisms.

The most significant ABCI interfaces that Υ must implement are Δ_{prepare}, Δ_{process}, and Δ_{finalize}. The consensus client $\mathcal{C}$ invokes each interface according to its internal specification and the current consensus state.

$$\mathcal{C} \rightleftarrows \left\{ \begin{array}{ll} \Delta_{\text{prepare}} & \text{construct block proposal} \\ \Delta_{\text{process}} & \text{validate proposed block} \\ \Delta_{\text{finalize}} & \text{commit finalized block} \\ \Delta_{\ldots} & \ldots \end{array} \right\} \rightleftarrows \Upsilon$$

An important aspect of CometBFT is that it establishes consensus on transactions, not blocks. This means that state changes must be perfectly reproducible from the agreed-upon list and order of transactions. To support Botanix-specific coordination logic, we reserve the first transaction slot in every block for **Non-Deterministic Data** (NDD), denoted γ. This special transaction is constructed by the block producer and executed by the **Foundation Layer** $\mathcal{F}$. It contains off-chain coordination messages that cannot be derived deterministically from prior state alone, but is still validated according to $\mathcal{F}$'s consensus rules. The last element in γ is always the **Foundation state proof** $\mathcal{F}^{\Phi}$, as defined in section 8.2.1:

$$\gamma_{|\gamma|} = \mathcal{F}^{\Phi}$$

The Foundation Layer is described in more detail in section 8.

2.1 Block Proposal Construction

When $\mathcal{C}$ decides to construct a block proposal by calling Δ_{prepare} on Υ, the endpoint combines two distinct sources: the Foundation Layer produces γ, while the standard Ethereum transaction pool $\mathcal{E}_{\text{pool}}$ provides user transactions. The resulting proposal always places γ first:

$$\lambda[\gamma, \mathbb{T}] \leftarrow \Upsilon.\Delta_{\text{prepare}} := \begin{cases} \gamma \leftarrow \mathcal{F} \\ \texttt{rollback}(\mathcal{F}) \\ \mathbb{T} \leftarrow \mathcal{E}_{\text{pool}} \\ \texttt{return } [\gamma, \mathbb{T}] \end{cases}$$

where $\mathbb{T} = [t_i]_{i=1}^{n}$

Note: `rollback()` discards tentative state changes and returns to the previous block's state. This allows validation to occur without persisting changes until finalization.

The function returns a CometBFT-constructed structure λ that includes additional consensus-related information (validator selection, evidence of equivocation, etc.), with the NDD γ at position zero followed by an ordered list of transactions $\mathbb{T}$, which may be empty. The EVM state proof $\mathcal{E}^{\Phi}_{\text{vm}}$ is placed in the structure's *appHash* field, following standard CometBFT convention. The Foundation state proof $\mathcal{F}^{\Phi}$ is embedded in the Ethereum

block's **extra data header**, making it part of $\mathcal{E}^{\Phi}_{\text{vm}}$. Consequently, both proofs are transitively committed in the *appHash*.

Formally:

$$\{\mathcal{E}^{\Phi}_{\text{vm}}, \mathcal{F}^{\Phi}\} \subseteq \lambda.\mathit{appHash}$$

2.2 Block Proposal Validation & Finalization

When $\mathcal{C}$ receives a block proposal $\lambda[\gamma, \mathbb{T}]$ from a peer, it submits the proposal to the Δ_{process} validation function of Υ. The validation process first extracts and validates γ (which must be present), passing it to the Foundation Layer for verification. The remaining transactions $\mathbb{T}$ are then passed to the Ethereum EVM $\mathcal{E}_{\text{vm}}$ for standard execution validation:

$$\left.\begin{array}{ll}\lambda[\gamma, \mathbb{T}] & \rightarrow \\ f & \leftarrow\end{array}\right\} \Upsilon.\Delta_{\text{process}} := \begin{cases}\gamma \rightarrow \mathcal{F} \\ \mathbb{T} \rightarrow \mathcal{E}_{\text{vm}} \\ \texttt{rollback}(\mathcal{E}_{\text{vm}}) \\ \texttt{rollback}(\mathcal{F}) \\ \texttt{return } f\end{cases}$$

$$\text{where } \mathcal{C} \text{ validates } \lambda.\texttt{appHash} \stackrel{?}{=} f.\texttt{appHash}$$

where f is a validation result indicating whether the proposal is accepted or rejected by the consensus rules. A block is only considered **canonical** (i.e., finalized) after $\mathcal{C}$ calls Δ_{finalize}, which replicates the behavior of Δ_{process} with the critical exception that state changes are now **committed** (persisted and canonical) rather than rolled back:

$$\left.\begin{array}{ll}\lambda[\gamma, \mathbb{T}] & \rightarrow \\ f & \leftarrow\end{array}\right\} \Upsilon.\Delta_{\text{finalize}} := \begin{cases}\gamma \rightarrow \mathcal{F} \\ \texttt{commit}(\mathcal{F}) \\ \mathbb{T} \rightarrow \mathcal{E}_{\text{vm}} \\ \texttt{commit}(\mathcal{E}_{\text{vm}}) \\ \texttt{return } f\end{cases}$$

2.3 System Flow: Pegin Lifecycle

When a user deposits Bitcoin into Botanix:

1. User constructs a Bitcoin transaction sending funds to a multisig address controlled by either a Tier 1 or Tier 2 pool
2. DynaFed routing logic (section 5.3) determines the destination based on:
   - Dynamic capacity target $H(t)$ (section 5.1)
   - Current collateralization levels of Tier 2 pools (5.2)
   - Risk-adjusted collateral availability (7)
3. Once the Bitcoin transaction achieves sufficient confirmations, the Foundation Layer registers it
4. The EVM layer mints corresponding tokens to the user's Botanix address

2.4 System Flow: Pegout Lifecycle

When a user withdraws Bitcoin from Botanix:

1. User burns tokens on the EVM layer via a dedicated smart contract, triggering a pegout request
2. The Foundation Layer records the pegout as **unassigned** with a list of qualified candidate pools
3. DynaFed's pegout allocation logic (section 5) selects which Tier 2 pool(s) should fulfill the request based on:
   - Collateralization deficits (priority to under-collateralized pools)
   - Load distribution across healthy pools
   - Risk assessment scores (section 7)
   - Fallback to Tier 1 pool if the amount exceeds funds
4. The selected pool's representative submits a **proposal** to the Foundation Layer (section 8.1), coupling the pegout with specific UTXOs
5. Pool members coordinate off-chain via FROST to sign the Bitcoin transaction
6. Once the transaction appears in a Bitcoin block and achieves finality depth, the Foundation Layer classifies it as **finalized**
7. The block tree pruning algorithm (section 9.2) automatically handles any Bitcoin forks

3 The FROST Challenge

The FROST threshold signature protocol presents a fundamental scalability challenge for federation design. DKG setup requires multiple rounds of package exchanges between all participants, creating significant coordination overhead. While DynaFed can schedule DKG processes asynchronously where timing is less critical, the two-round signing protocol for pegout fulfillment remains a bottleneck, with larger federations facing increased message complexity, higher failure rates, and greater coordination burden.

This creates a tension between security and operational efficiency. Larger federations provide stronger security guarantees but struggle with FROST's coordination requirements. Smaller federations operate more efficiently

but offer weaker security and are more vulnerable to attacks.

To address this tradeoff, we propose a two-tier federation system as described in section 4. Tier 1 serves as a large "cold wallet" responsible for holding the majority of funds with maximum security. Tier 2 operates as a smaller "hot wallet" optimized for responsive pegout fulfillment. This separation allows each tier to play to its strengths: Tier 1 prioritizes security through size, while Tier 2 prioritizes operational efficiency through agility.

See appendix 9.1 for a detailed breakdown of message complexity.

4 Two-Tier Federations

We generally want funds to be as secure as possible, which suggests keeping them in the largest federation practical. However, larger federations have drawbacks: increased message complexity, reduced flexibility, and higher failure rates for operations like DKG processes and pegout signing requests as expanded on in section 3.

Fortunately, we can leverage a key observation: the majority of funds remain dormant with minimal movement, while only a small subset is actively used for pegouts. This allows us to structure a system that addresses both security and operational efficiency requirements:

- **Tier 1** serves as the larger "cold wallet" where the majority of funds remain secured
- **Tier 2** serves as the smaller "hot wallet" responsible for honoring pegouts

DynaFed ensures that Tier 2 funds are substantially collateralized on Tier 1, *ideally* at 100%. This collateralization model provides several benefits:

- **Reduced risk**: Tier 2 failures have limited impact due to collateral backing
- **Increased flexibility**: We can experiment with different federation sizes, collateralization requirements, and rotation schedules for Tier 2
- **Lower barriers**: We can accept newer or less-established CometBFT validators into Tier 2

Note: While we generally refer to Bitcoin multisigs as federations, we'll adopt the term **pools** going forward, which aligns better with how the broader crypto and DeFi ecosystem conceptualizes fund movements and custody.

4.1 Tier 1 Architecture

Tier 1 serves as the primary cold storage federation, holding the majority of bridged funds with maximum security guarantees. DynaFed maintains **exactly one active Tier 1 federation** under normal operation, though multiple Tier 1 pools may temporarily coexist during migration periods when an old pool is being sunset in favor of a new pool.

The Tier 1 federation is intentionally large and conservative in its membership. DynaFed selects the most reliable, established, and senior members of the CometBFT validator set for this critical role. This selection bias reflects the security-first mandate of cold storage:

- **High barriers to entry**: New validators face significant hurdles to joining Tier 1, requiring demonstrated reliability and stake history
- **Membership stability**: Established Tier 1 members are rarely removed unless they exhibit clear unreliability or misconduct
- **Reputation weighting**: Long-standing validators with proven track records receive priority over newer participants

This conservative approach prioritizes security and stability over operational flexibility, which is appropriate given that Tier 1 holds the bulk of user funds. Validators seeking to participate in the federation typically begin in Tier 2 pools, where they can establish their reliability before being considered for Tier 1 membership.

4.2 Tier 2 Architecture

Tier 2 pools serve as operational hot wallets, holding a smaller subset of funds optimized for fulfilling pegout requests with minimal latency. These pools are intentionally smaller than Tier 1, allowing for more flexible federation management including faster rotation schedules, lower barriers to entry for validators, and more dynamic collateralization requirements.

This proposal assumes the existence of **multiple Tier 2 pools**, and DynaFed is designed to manage routing and collateralization across all active Tier 2 pools simultaneously. This multi-pool architecture provides several benefits:

- **Load distribution**: Pegout requests can be distributed across multiple pools, reducing the operational burden on any single federation
- **Risk diversification**: Exposure is distributed across multiple Tier 2 pools, limiting the impact if any single pool's funds exceed its members' staked collateral.

- **Gradual migration**: New pools can be introduced while existing pools are phased out without service interruption
- **Specialization potential**: Different pools could eventually serve different use cases or security profiles

In practice, especially during initial deployment, there may be only a single active Tier 2 pool at any given time (excluding transitionary periods during pool migration). However, the protocol is designed from the ground up to handle multiple concurrent pools, ensuring the architecture can scale as network demand grows.

4.2.1 Future Extensibility

This architecture also enables a future extension where Tier 2 could become truly open to external service providers and third parties who are not part of the CometBFT validator set. However, such external participants would be subject to a mandatory 100% collateralization requirement.

4.3 Staking, Deposits & Collateralization

Botanix is a Proof-of-Stake (PoS) network that follows the core concepts and mechanisms that have become standardized across the PoS ecosystem. Staking, the practice of locking funds that can be penalized ("slashed") for misbehavior, forms the core of the network's security model. It serves dual purposes: determining who has the right to participate in network consensus, and making attacks economically expensive, if not practically impossible.

Unlike many PoS networks, Botanix does not have its own floating cryptocurrency. Instead, users deposit Bitcoin into the network, which can be minted and redeemed at a 1:1 ratio. These minted coins function similarly to native tokens in other PoS networks, enabling both economic activity and consensus participation.

Users can utilize minted coins for transactions and smart contract interactions in their liquid form. However, to gain voting rights for consensus-critical operations, coins must be explicitly **staked**. This staking process locks coins for a specific duration, rendering them illiquid. Conversely, **unstaked** coins remain liquid from a Botanix consensus perspective, though they may still be subject to smart contract-specific restrictions.

The distinction between **unstaked** and **staked** funds - or equivalently, liquid and illiquid funds - directly determines:

- How collateralization levels are calculated for multisig federations
- How DynaFed directs pegins to appropriate federations
- Which multisig federation is responsible for honoring pegouts

This relationship is described in detail in the Pool Management section 5.

5 Pool Management

This section describes how DynaFed processes pegins and pegouts across the two-tier federation system introduced in section 4. The core principle is straightforward: Tier 2 hot wallets should contain just enough funds to manage pegouts efficiently, while the Tier 1 cold wallet should minimize fund movements for maximum security.

To achieve this balance, DynaFed employs several interconnected mechanisms:

- **Dynamic Capacity Target** (section 5.1): An adaptive estimate of how much liquidity Tier 2 pools need, adjusting to recent pegout activity while filtering out short-term noise.
- **Collateralization Level** (section 5.2): A measure of each Tier 2 pool's backing relative to its held funds, incorporating risk-adjusted stake from pool members to determine over- or under-collateralization.
- **Pegin Allocation Logic** (section 5.3): Routing rules that direct incoming deposits to the appropriate pool, prioritizing over-collateralized Tier 2 pools while respecting the dynamic capacity target.
- **Pegout Allocation Logic** (section 5.4): Selection criteria for which pool(s) should fulfill withdrawal requests, prioritizing deficit recovery in under-collateralized pools before distributing load across healthy pools.

Together, these mechanisms ensure that Tier 2 pools remain adequately funded and collateralized for responsive pegout fulfillment, while excess funds flow to the more secure Tier 1 pool.

5.1 Dynamic Capacity Target

We use an exponentially weighted moving average (EWMA) with a smoothing factor to determine the dynamic capacity target for Tier 2 pools. This approach is inspired by TCP congestion control algorithms, which use similar techniques to estimate network conditions while filtering out transient spikes and noise.

The EWMA formula is computationally efficient and requires minimal historical data retention, making it

ideal for DynaFed to dynamically adjust the appropriate amount of funds to maintain in Tier 2 wallets. The target naturally increases during periods of high pegout activity and decreases during quieter periods, all while smoothing out unexpected network spikes.

Specifically, we define the **dynamic capacity target** as:

$$H(t) = \alpha \times P(t) + (1 - \alpha) \times H(t - 1) \tag{1}$$

where:

- $H(t)$ is the dynamic capacity target at time t (block number)
- $P(t)$ is the recent pegout volume (e.g., total pegout value over the last measurement period)
- α is the smoothing factor, controlling the balance between responsiveness and stability

The smoothing factor α typically ranges from 0.1 to 0.3 for stable behavior, though higher values can be used when faster adaptation to changing pegout patterns is desired. In practice, the dynamic capacity target should also be bounded by a floor and ceiling, expressed as a percentage of total bridged funds, to prevent extreme values during periods of unusual activity.

5.2 Collateralization Level

When DynaFed selects M candidates to construct a Tier 2 pool, it applies a bias toward reliable and well-functioning members with larger amounts of staked funds as described in section 7. The total sum of all risk-adjusted staked funds across every member is considered the **total collateralization** of the Tier 2 pool M, denoted as:

$$C_M = \sum_{m \in M} (S_m \times \phi(r_m)) \tag{2}$$

where S_m represents the staked funds of member m and $\phi(r_m)$ is the **risk adjustment factor** in the range $[0, 1]$ as defined in equation 7.

A newly established Tier 2 pool initially starts with zero funds, denoted as $F_M = 0$, and gradually receives pegins allocated by DynaFed. As funds accumulate, we can compare the total funds against the collateralization level. When $F_M < C_M$, the pool is over-collateralized with a **collateralization surplus**. Conversely, when $F_M > C_M$, the pool is under-collateralized with a **collateralization deficit**.

As noted in section 4, DynaFed attempts to maintain over-collateralization for all Tier 2 pools. However, this depends on DynaFed's ability to select M candidates whose total collateralization exceeds the dynamic capacity target described in section 5.1, which cannot be guaranteed. Since DynaFed mandates that at least one Tier 2 pool must be active at all times, **it does allow under-collateralized pools to exist**. In such cases, the security model falls back on the assumption that the majority of selected members are trustworthy. We consider this an acceptable trade-off given the operational requirements.

5.3 Pegin Allocation Logic

DynaFed determines the appropriate destination for a pegin p based on the resulting collateralization state after allocation. For a given Tier 2 pool, the post-allocation funds would be $F_M + p$, and we can evaluate:

$$\Delta_{M,p} = C_M - (F_M + p) \tag{3}$$

DynaFed follows this priority order when allocating pegins:

Priority 1: Select the Tier 2 pool that remains most over-collateralized after receiving the pegin. This distributes deposits toward pools with greater collateral headroom.

Priority 2: If no Tier 2 pool can maintain or achieve over-collateralization, DynaFed evaluates whether the pegin should be directed to the Tier 1 pool instead:

$$\left(\sum_{M \in \text{Tier2}} F_M \right) + p > H(t) \tag{4}$$

This checks if the total Tier 2 funds plus the new pegin would exceed the dynamic capacity target $H(t)$ at time t, as defined in section 5.1. If yes, the pegin should be **directed to the Tier 1 pool**. If no, DynaFed selects the Tier 2 pool with the lowest collateralization, minimizing the under-collateralization risk.

This multi-tier decision process balances security (preferring over-collateralization), operational requirements (maintaining active hot wallets), and risk management (minimizing exposure when under-collateralization is unavoidable).

5.4 Pegout Allocation Logic

Similar to pegin allocation logic described in section 5.3, DynaFed uses collateralization levels to determine which Tier 2 pool should honor pegout requests. However, pegouts offer more flexibility than pegins: while pegins require a single destination address for users to send funds to, pegouts can be fulfilled from multiple sources. DynaFed may select multiple Tier 2 pools as **candidates** to collectively honor pegout requests.

For a given Tier 2 pool fulfilling one or more pegouts totaling p, the post-fulfillment funds would be $F_M - p$. We can evaluate the resulting collateralization state:

$$\Delta_{M,p} = C_M - (F_M - p) = C_M - F_M + p \quad (5)$$

Note that fulfilling a pegout *increases* the collateralization surplus (or decreases the deficit) since funds are leaving the pool while the staked collateral remains constant.

DynaFed follows this priority order when selecting candidate pools for pegout fulfillment:

Priority 1 - Deficit Recovery: If any Tier 2 pool has a collateralization deficit, DynaFed allocates pegouts exclusively to that pool to reduce the deficit as quickly as possible. If multiple pools have deficits, DynaFed selects the pool with the largest deficit, prioritizing the highest-risk pool for immediate rebalancing.

Priority 2 - Distributed Fulfillment: If all Tier 2 pools maintain a collateralization surplus, DynaFed selects all pools as candidates, distributing the pegout load across the system.

Currently, Botanix does not charge a service fee for pegout fulfillment beyond the standard Bitcoin L1 network fee shared proportionally among all pegout recipients. The protocol assumes that candidates will honor pegouts in good faith as part of their validator duties. In the future, **an additional service fee could be introduced** to create explicit economic incentives for timely pegout fulfillment, integrating it more directly into the validator reward mechanism.

5.4.1 Future Enhancements

This allocation logic may be extended with additional metrics such as:

- **Pool reliability scores** based on historical fulfillment performance
- **Response time tracking** to prioritize faster candidates
- **Dynamic candidate selection** that gradually excludes consistently unreliable pools

These enhancements would improve the robustness of the pegout system while maintaining the core collateralization-based security model.

6 Bitcoin Fork Awareness

Pegout proposals must be tracked across the Bitcoin network to enable the entire Botanix validator set to deterministically establish when a pegout proposal has reached probabilistic finality, been replaced by an updated proposal, or been discarded due to a chain reorganization. This is achieved by **registering** both the transaction and the Bitcoin block in which it appears.

The verification process operates in two stages:

- **Block Header Verification**: The Bitcoin block header is verified by computing its hash and confirming that it falls below the Proof-of-Work target. The block's ancestry is validated by checking that its parent block has already been registered in the Foundation Layer, ensuring the block is part of a valid chain.
- **Transaction Inclusion Verification**: A Merkle inclusion proof demonstrates that the transaction exists within the registered block. This proof is verified against the Merkle root stored in the block header, cryptographically confirming transaction inclusion without requiring full block data.

This dual verification mechanism enables the Foundation Layer to track Bitcoin chain state in a lightweight, deterministic manner. Validators independently verify pegout finality by observing registered blocks and their depth, handling Bitcoin forks and reorganizations without external oracles or trusted data feeds.

6.1 Block Tree Integration

The Foundation Layer maintains a **block tree** structure responsible for tracking Bitcoin chain state and resolving competing forks. The block tree exposes a single operation (specified in section 9.2.2):

$$\Pi(b_{\text{new}}) \to \{\text{fate}(b_1), \text{fate}(b_2), \ldots\} \quad (6)$$

This operation inserts a new block b_{new} into the tree and returns a set of pruned blocks with their classifications. Each $\text{fate}(b_i)$ indicates whether block b_i has been:

- **Finalized**: Part of the canonical chain with sufficient confirmation depth
- **Orphaned**: Part of a rejected fork due to chain reorganization

The returned set may be empty if no blocks are pruned during the insertion. The complete block tree specification, including pruning criteria and classification logic, is detailed in section 9.2.

7 Risk Assessment

As discussed in section 5, DynaFed leverages collateralization levels and dynamic capacity targets when allocating pegins and assigning pegouts. However, these mechanisms should also influence pool lifecycle management itself. We employ a selection bias favoring reliable, well-functioning pools while disadvantaging under-performers

and **eventually sunsetting problematic pools**. The top priority is fund security, requiring swift action when a pool experiences member attrition or operational degradation.

We apply this bias through a simple sigmoid risk adjustment factor $\phi(r)$ that modulates a pool's effective collateralization. Following a *"first gradually, then suddenly"* approach, we exercise caution when allocating to newly established pools and when determining sunset timing for failing pools. The function $\phi(r)$ produces a value in $[0, 1]$ that scales the nominal collateralization level C_M as defined in section 5.2, yielding an effective collateralization of $C_M \times \phi(r_M)$. When $\phi(r_M) = 1$, the pool operates at full capacity with complete collateralization leverage. When $\phi(r_M) = 0$, the pool is effectively defunct and must be sunset.

We define the **risk adjustment factor** as:

$$\phi(r) = \frac{1}{1 + e^{k(r - r_{mid})}} \tag{7}$$

and visualized in figure 2, where $k = 15$ and $r_{mid} = 0.5$. We employ a balanced sigmoid curve without excessive parameter tuning, as the primary complexity lies in computing r, the **risk level** as defined in section 7.1.

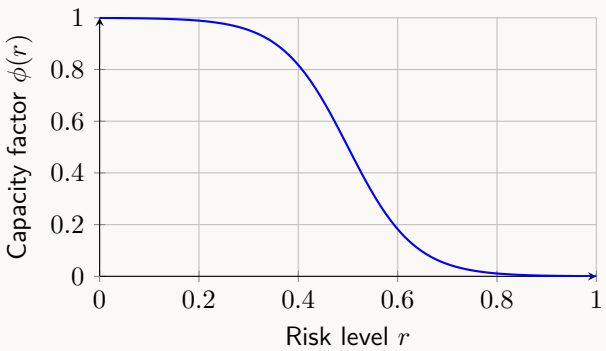


Figure 2: Sigmoid capacity function $\phi(r)$ modulating effective collateralization based on pool risk level

7.1 Risk Level

The risk level r_M quantifies the operational health of pool M by assessing the participation and reliability of its members, with a preference toward established, proven validators. This metric feeds directly into the capacity adjustment function $\phi(r_M)$ defined in equation 7, determining the pool's effective collateralization.

DynaFed tracks individual member weights and aggregates them to compute the **pool-level risk**:

$$r_M = 1 - \frac{\sum_{m \in M} w(m)}{|M|} \tag{8}$$

where $w(m)$ is the **member weight** for member m, defined as:

$$w(m) = \sigma_M \times \psi_M \times (\alpha \times s_m + (1 - \alpha) \times t_m) \tag{9}$$

with components:

- $\sigma_M \in \{0, 1\}$: state control parameter as defined in section 7.1.1
- $\psi_M \in \{0, 1\}$: emergency controller as defined in section 7.1.2
- $\alpha \in [0.5, 0.7]$: weight parameter balancing recent performance versus long-term tenure
- s_m: participation score as defined in section 7.1.3
- t_m: tenure score as defined in section 7.1.4

7.1.1 State Control Parameter

The state control parameter σ_M acts as an activation gate that prevents premature capacity allocation during the observation window when block samples are insufficient for reliable member scoring. Newly established pools require time to accumulate sufficient data before their participation and tenure scores can meaningfully reflect operational health.

The state control parameter is defined as:

$$\sigma_M = \begin{cases} 0 & \text{if } b - b_0^M < N_{\min} \text{ or } p_M < |M| \\ 1 & \text{if } b - b_0^M \geq N_{\min} \text{ and } p_M = |M| \end{cases} \tag{10}$$

where:

- b: current block height
- b_0^M: block height when pool M was created (after DKG completion)
- $N_{\min}$: minimum observation period (e.g., 100 blocks $\approx$ 10 minutes)
- $p_M = |\{m \in M : s_m(b) > 0\}|$: count of members who have submitted at least one FROST proof, where s_m is participation score as described in section 7.1.3.

During the observation window ($\sigma_M = 0$), all member weights become zero regardless of their individual participation or tenure scores. This suppresses the influence of high volatility in both s_m and t_m that occurs when few blocks have elapsed. The pool remains in this dormant state until sufficient time has passed to establish reliable scoring patterns.

Once σ_M transitions to 1, the pool is considered **activated** and its risk level directly reflects member performance through the standard risk calculation. Conceptually, a pool is being **onboarded** when $\sigma_M = 0$ during

the observation phase, which forces $r_M = 1$ regardless of individual member scores. A pool is being **offboarded** or sunset when $\sigma_M = 1$ but $r_M \to 1$ due to member degradation, triggering capacity wind-down through the sigmoid function. This binary state transition ensures a clean separation between the observation phase and normal operational status.

7.1.2 Emergency Controller

The emergency controller ψ_M provides a critical safety mechanism that forces immediate pool shutdown when the number of active members approaches the FROST signing threshold. While the gradual risk assessment through $\phi(r_M)$ handles normal operational degradation, FROST threshold signatures require a minimum number of functioning members to execute transactions. If a pool degrades to the point where it risks falling below this operational threshold, gradual wind-down is insufficient; the pool must be sunset immediately before it becomes unable to sweep its own funds.

The emergency controller is defined as:

$$\psi_M = \begin{cases} 0 & \text{if } A_M < \lceil |M| \times \rho_{\text{safety}} \rceil \\ 1 & \text{otherwise} \end{cases} \tag{11}$$

where:

- $A_M = |\{m \in M : s_m(b) > \theta_{\text{active}}\}|$: count of members exceeding the liveness threshold
- θ_{active}: liveness threshold for considering a member "active" (e.g., 0.8)
- $\rho_{\text{safety}} = \frac{t_M}{|M|} + \epsilon_M$: safety ratio relative to FROST threshold
- t_M: FROST signing threshold for pool M (e.g., 12 for a 12-of-16 multisig)
- ϵ_M: collateralization-aware safety buffer

The safety buffer ϵ_M adapts based on pool tier and collateralization status:

$$\epsilon_M = \begin{cases} \epsilon_{\text{strict}} & \text{if Tier 1 or } F_M > C_M \\ 0 & \text{if Tier 2 and } F_M \leq C_M \end{cases} \tag{12}$$

where $\epsilon_{\text{strict}} = 0.125$ provides a safety buffer above the FROST threshold as a fraction of pool size, and F_M and C_M are the pool's current funds and total collateralization as defined in section 5.2.

Design Rationale: Tier 1 pools and under-collateralized Tier 2 pools maintain strict safety buffers because fund loss is possible if the pool becomes inoperable. Over-collateralized Tier 2 pools use $\epsilon_M = 0$, triggering emergency shutdown only at the actual FROST threshold. This relaxed approach is justified because these pools have full collateral backing on Tier 1. The worst-case failure is service degradation (pegout delays), not fund loss. This provides maximum operational flexibility for hot wallets while the collateral guarantee provides the safety net.

When $\psi_M = 0$, all member weights become zero regardless of individual scores, forcing $r_M = 1$ and consequently $\phi(r_M) \approx 0$. This immediately halts new pegin allocation to the pool and triggers urgent fund sweeping to healthy pools.

7.1.3 Participation Score

The participation score s_m measures the recent liveness and responsiveness of member m. Members prove their active participation by submitting FROST signature proofs via the NDD as described in section 8. These proofs serve dual purposes: they demonstrate that members possess their key material and are online, actively participating in consensus.

A critical constraint is that FROST proof submission requires the member to be selected as block producer, enabling them to include their proof in the NDD. This means members compete for block production slots not just among pool members $|M|$, but across the entire validator set N_v. We assume CometBFT's fair block producer selection guarantees that over sufficient time, every validator has approximately equal opportunity to produce blocks. Members may implement external gossiping mechanisms allowing others to include their proofs on their behalf, which is secure given that FROST proofs are cryptographically signed.

The participation score uses an exponentially weighted moving average (EWMA), similar to the dynamic capacity target in section 5.1. This approach is computationally efficient, requires minimal storage, and naturally adapts to changing behavior:

$$s_m(b) = \beta \times p_m(b) \times N_v + (1 - \beta) \times s_m(b - 1) \tag{13}$$

where:

- $s_m(b)$: participation score at block height b
- $p_m(b) \in \{0, 1\}$: proof indicator (1 if member m submitted valid FROST proof in block b, 0 otherwise)
- N_v: total number of validators in the network
- $\beta \in [0.001, 0.01]$: smoothing factor controlling responsiveness versus stability

The N_v scaling factor normalizes for expected block production frequency, ensuring that members who consistently submit proofs when selected converge to $s_m \approx 1$,

while unreliable members decay toward $s_m \approx 0$. The initial condition is $s_m(b_0^M) = 0$, requiring new members to establish their reliability through observed behavior.

The smoothing factor β is intentionally small because the EWMA updates every block (every 6 seconds), while individual members produce blocks infrequently at rate $1/N_v$. This ensures the score smooths over multiple block production opportunities rather than reacting to single events.

7.1.4 Tenure Score

The tenure score t_m measures how long a member has been part of an active pool, reflecting establishment and stability within the system. Unlike the participation score which tracks recent performance, tenure captures long-term presence independent of current behavior. This distinction ensures that newly activated members with perfect participation still operate at reduced weight until they prove sustained reliability over time.

Tenure is calculated as the fraction of time elapsed since pool activation, capped at the maturity threshold:

$$t_m = \min\left(1, \frac{b - b_0^M}{N_{\text{maturity}}}\right) \tag{14}$$

where:

- t_m: tenure score for member m
- b: current block height
- b_0^M: block height when pool M was activated
- N_{maturity}: maturity threshold (e.g., 14,400 blocks $\approx$ 24 hours)

All members of pool M share the same tenure score since it depends only on pool activation time, not individual member behavior. A newly activated pool begins with $t_m = 0$ for all members, reaching full tenure credit ($t_m = 1$) after N_{maturity} blocks of operation. This creates the "gradually, then suddenly" capacity ramp described in section 7, where new pools start with limited effective collateralization that grows as they demonstrate sustained operational stability.

The tenure score is computationally trivial, requiring only two block height values and basic arithmetic, and provides the timespan that complements the performance-based participation score in the composite member weight.

8 The Foundation Layer

The Foundation Layer is the central coordination component that orchestrates Botanix's multisig federation system. It integrates several subsystems described throughout this paper:

- **Pool Management** (section 5): Pegin and pegout allocation across the two-tier federation architecture
- **Risk Assessment** (section 7): Pool health monitoring and lifecycle management
- **Bitcoin Fork Awareness** (section 6): Tracking pegout proposals across Bitcoin's probabilistic finality model
- **State Commitments** (section 8.2): Cryptographic proofs enabling deterministic validation across all validators

As described in section 2, the Foundation Layer produces the **Non-Deterministic Data (NDD)** γ included in every block. While the NDD is constructed subjectively by the block producer, all validators verify its contents against the Foundation Layer's consensus rules. This design enables off-chain coordination, such as FROST signing rounds and pegout batching, to be anchored on-chain without requiring validators to track external state.

The NDD supports the following message types:

- **Pegout Proposal** (section 8.1): Submits a new proposal, upgrades an existing proposal (section 8.1.1), or cancels a proposal (section 8.1.2). This message type also covers fund sweeping during pool migration and sunsetting operations.
- **Bitcoin Header** (section 6): Registers a new Bitcoin block header, validated by verifying that its hash falls below the Proof-of-Work target and that its parent is already known to the Foundation Layer.
- **Bitcoin Transaction** (section 6): Associates a pegout proposal with a registered Bitcoin block via a Merkle inclusion proof, enabling finality tracking.
- **FROST Signature Proof** (section 7.1.3): Demonstrates that a pool member possesses valid key material and is actively participating in consensus. These proofs feed into the participation score used for risk assessment.

Note: The NDD message format is designed for extensibility. Future protocol versions may introduce additional message types for staking, slashing, and other consensus-critical operations.

The Foundation Layer operates on a critical principle: it treats all input as potentially malicious, verifies input through cryptographic proofs, and makes decisions based solely on provided inputs without requiring access to external networks. This self-contained verification model ensures that validators reach consensus on Foundation state without relying on external oracles or subjective observations of the Bitcoin network.

8.1 Pegout Proposals

Bitcoin uses a probabilistic finality system rather than deterministic finality. We assume that after sufficient time and spent resources, a reorganization becomes economically unviable if not practically impossible. However, given this probabilistic nature and competing blocks, the Foundation Layer must be fork-aware and consider various scenarios:

- A Bitcoin transaction can be rejected by nodes or miners even though it's valid under consensus rules. For example, if fee rates are unattractive, local mempool policies differ, or other configurable parameters aren't met. Different nodes and miners may have conflicting views on which transactions to propagate or include.
- A previously rejected transaction might later be picked up and propagated, potentially remaining unconfirmed for extended periods.
- A Bitcoin transaction might need to be accelerated or replaced by DynaFed. There's no guarantee that a replacement transaction will take precedence over the original.
- Even if a transaction is widely propagated, miners may choose not to include it in a block for arbitrary reasons. Fundamentally, there's no way to deterministically or objectively determine whether a Bitcoin transaction will be mined.

In our Foundation Layer model, **UTXO reuse** becomes the key safeguard that anchors our entire system, especially when integrated with the TEE/TEM architecture. This dramatically reduces state management complexity. Pegouts are categorized in one of two states:

- **Unassigned**: Fresh pegouts that have been triggered on the Botanix EVM but not yet included in any proposal.
- **Proposed**: Pegouts that are part of a proposed Bitcoin transaction, coupled with the selected UTXO(s).

Critically, a pegout proposal must be submitted to the Foundation Layer before multisig members sign the PSBT. The coordinator or representative of a multisig federation must first include their proposal in the Foundation Layer, and only then will other members accept signing the proposal off-chain. That proposal remains in the Proposed state until it's finalized, or gets removed by a competing transaction being finalized first.

The Foundation Layer enforces a crucial invariant: **pegouts and UTXOs in a proposal are tightly coupled**. The layer will only allow replacing or updating proposals if the updated (competing) proposal reuses at least one UTXO from the original. This means that if multiple Bitcoin transactions all share at least one UTXO, we have a core guarantee that **only one will become probabilistically finalized**, the others will fail due to the spent UTXO. This elegant property handles forks, replacements, and race conditions without complex state tracking.

8.1.1 Updating Proposal

Federation representatives may need to update an existing pegout proposal to adjust transaction parameters such as fee rates, consolidate additional pegouts, or respond to changing mempool conditions. The Foundation Layer supports proposal updates through a replacement mechanism anchored by UTXO reuse.

To update a proposal, the representative submits a new proposal that satisfies two requirements:

- **UTXO continuity**: The new proposal must reuse at least one UTXO from the original proposal, establishing a link between the two versions
- **Federation authorization**: Only the original proposing federation can submit updates, verified through the federation identifier

This mechanism enables common operations such as:

- **Fee adjustment**: Replacing a low-fee transaction with a higher-fee version (Replace-By-Fee)
- **Pegout consolidation**: Including additional pending pegouts in an updated transaction
- **UTXO optimization**: Adjusting input selection for better transaction efficiency

The UTXO reuse property ensures that only one version of the proposal can eventually finalize on the Bitcoin blockchain. When the updated transaction confirms, the original proposal is automatically invalidated through the spent UTXO, preventing any possibility of double-execution. The Foundation Layer tracks both proposals until one achieves finality, at which point the competing version is removed from state.

8.1.2 Cancelling Proposal

Circumstances may require canceling a pegout proposal without fulfilling its original pegouts. The Foundation Layer supports cancellation through the same UTXO reuse mechanism used for updates, with two primary approaches:

- **Consolidation**: The federation submits a transaction that spends the proposal's UTXOs back to an

address it controls, without including any pegout outputs. This effectively "consumes" the UTXOs while returning funds to the federation.

- **Replacement**: The federation submits an entirely different proposal that reuses at least one UTXO from the original. When the replacement achieves finality, the original proposal is invalidated through the spent UTXO.

When the cancellation transaction achieves finality, the Foundation Layer automatically returns the original pegouts to the unassigned state, making them available for inclusion in future proposals. This recovery mechanism ensures that user withdrawal requests are never permanently lost due to proposal failures. They simply re-enter the queue for the next available federation to fulfill.

8.2 Foundation State Proof

The Foundation Layer maintains a cryptographic proof of its complete state, enabling deterministic validation across all consensus participants. This state proof commits to four key components: context metadata, the commitment trie root, tracked Bitcoin headers, and auxiliary events that enable efficient state indexing and reconstruction.

The commitment trie root r_{trie} (section 8.2.1) employs the Base-16 Modified Patricia Merkle Tree as implemented by Parity Technologies[1] to construct and validate state proofs. The architecture separates data storage from commitment construction: the trie operates exclusively on cryptographic hashes of key-value pairs, maintaining independence from the underlying database implementation.

The system supports any key-value database backend (e.g., RocksDB, MDBX) that provides atomic commit and rollback mechanisms. These transactional guarantees must be supplied by the database layer and fall outside the scope of the trie commitment specification itself.

8.2.1 State Proof Structure

The complete Foundation state proof is represented as:

$$\mathcal{F}^{\Phi} = (\text{context}, r_{\text{trie}}, \text{headers}, \text{events}) \tag{15}$$

where:

- context: Protocol context and version metadata (section 8.2.2)
- r_{trie}: 32-byte commitment trie root as produced by the Tree implementation
- headers: Set of tracked Bitcoin block hashes
- events: Sequence of auxiliary events (section 8.2.3) for state reconstruction

8.2.2 Context Metadata

The context provides version information and is primarily reserved for future protocol extensions:

$$\text{context} = (\text{height}) \tag{16}$$

where height is the current Botanix block height.

8.2.3 Auxiliary Events

Auxiliary events provide an efficient event log enabling state reconstruction and indexing without reprocessing the entire commitment validation sequence.

The following event types are recorded:

- **Initiated Pegout**: Records a newly initiated pegout available for spending, including the pegout identifier and the list of candidate federations qualified to fulfill it.
- **Submitted Proposal**: Records a newly submitted pegout proposal, capturing the complete proposal entry.
- **New Bitcoin Header**: Records a Bitcoin block header being tracked by the Foundation Layer.
- **Registered Bitcoin Transaction**: Records a Bitcoin transaction registration, associating a transaction with its containing block and the set of pegouts it fulfills.
- **Finalized Bitcoin Header**: Records a finalized Bitcoin block along with the set of pegout identifiers that have reached finality.
- **Orphaned Bitcoin Header**: Records an orphaned Bitcoin block along with the set of pegout identifiers that have been returned to unassigned (spendable) state.

[1] `https://github.com/paritytech/trie`

9 Appendix

This appendix provides technical details and derivations referenced throughout the main text.

9.1 DKG: Message Complexity

This appendix provides a detailed breakdown of the message complexity for FROST DKG setup, as referenced in section 3. We measure the number of **messages that must be sent over the wire**, where the coordinator functions not only as a participant in the multisig setup, but also as a message forwarder/router.

We examine two approaches: **with and without message bundling**. The key difference is that with bundling, the coordinator waits until all messages per round are received, then sends all necessary packages to each non-coordinator as a single bundle. This applies to both received and sent messages in round 2. Without bundling, the coordinator forwards each package individually as they are received.

9.1.1 Comparison

Bundling reduces message complexity from quadratic $O(n^2)$ to linear $O(n)$:

- **Without bundling**: $3n^2 - 5n + 2$ messages
- **With bundling**: $4n - 4$ messages

For example, with $n = 32$ participants:

- Without bundling: 2,914 messages
- With bundling: 124 messages

With $n = 64$ participants:

- Without bundling: 11,970 messages
- With bundling: 252 messages

And with $n = 128$ participants:

- Without bundling: 48,514 messages
- With bundling: 508 messages

Bundling represents a **significant reduction** in message count, though at the cost of larger individual message sizes.

9.1.2 With Bundling

The coordinator waits until all messages per round are received, then sends all necessary packages to each non-coordinator as a **single bundle**. This applies to both round 1 and round 2.

Each Round Each non-coordinator sends their package to the coordinator. Since the coordinator already has their own package, they receive the following number of messages over the wire:

$$n - 1$$

The coordinator then forwards all packages as a single bundle to each of the $n - 1$ non-coordinator member (excluding themselves), sending:

$$n - 1$$

Total messages per round:

$$(n - 1) + (n - 1) = 2n - 2$$

Total The total message complexity across both round 1 and round 2:

$$\begin{aligned} &2 \times (2n - 2) \\ &= 4n - 4 \end{aligned} \tag{17}$$

9.1.3 Without Bundling

The coordinator forwards each package individually rather than bundling them together. While this keeps individual message sizes smaller, it significantly increases the total message count.

Round 1 Each non-coordinator sends their package to the coordinator. Since the coordinator already has their own package, they receive the following number of messages over the wire:

$$n - 1$$

The coordinator forwards each of the $n - 1$ packages to each of the $n - 1$ non-coordinators (excluding themselves), sending:

$$(n - 1)(n - 1) = n^2 - 2n + 1$$

Total messages for round 1:

$$(n - 1) + (n^2 - 2n + 1) = n^2 - n$$

Round 2 Each non-coordinator creates individual packages for each other member (excluding themselves), sending $n-1$ packages to the coordinator. Since the coordinator already has their own packages, they receive the following number of messages over the wire:

$$(n-1)(n-1) = n^2 - 2n + 1$$

The coordinator then forwards each individual package to its corresponding recipient (excluding themselves), sending:

$$(n-1)(n-1) = n^2 - 2n + 1$$

Total messages for round 2:

$$(n^2 - 2n + 1) + (n^2 - 2n + 1) = 2n^2 - 4n + 2$$

Total The total message complexity across both round 1 and round 2:

$$\begin{aligned}(n^2 - n) + (2n^2 - 4n + 2) \\ = 3n^2 - 5n + 2\end{aligned} \tag{18}$$

9.2 Block Tree

The block tree maintains a lightweight representation of the Bitcoin blockchain, tracking competing forks and automatically pruning blocks as they reach sufficient confirmation depth. This structure enables deterministic fork resolution across all Botanix validators without requiring full Bitcoin node synchronization.

The block tree serves two critical functions:

- **Fork Detection**: Identifies competing Bitcoin chains and preserves unresolved forks until one becomes canonical
- **Automatic Pruning**: Removes blocks that have either reached finality or been orphaned, preventing unbounded memory growth

The theoretical motivation for Bitcoin fork awareness and its role in pegout finality is described in section 6. This section provides the technical specification for the block tree data structure and its operations.

9.2.1 Core Definitions

Let B denote the set of all blocks in the tree, where each block $b \in B$ has:

- A block hash $h(b)$
- A parent hash $p(b)$
- A set of children $C(b) \subseteq B$
- A relative height $r(b) \in \mathbb{N}$, defined as the distance from the tree's root (elder)

We define:

- $T \subseteq B$: the set of tips (blocks with no children, i.e., $C(b) = \emptyset$)
- $e \in B$: the elder (oldest retained block with $r(e) = \min_{b \in B} r(b)$)
- $h_{best} = \max_{b \in B} r(b)$: the best (maximum) height in the tree
- d_{conf}: the confirmation depth parameter (minimum 3)

9.2.2 Block Insertion and Pruning

The block insertion operation processes a new Bitcoin block and automatically prunes finalized and orphaned blocks:

$$\Pi(b_{new}) \to \{\text{fate}(b_1), \text{fate}(b_2), \ldots\} \tag{19}$$

This operation inserts the new block, performs two-phase pruning, and returns the possibly empty set of pruned blocks with their classifications as defined in section 9.2.6.

When inserting a new block b_{new} with parent b_{parent}:

1. Height assignment:

$$r(b_{new}) = r(b_{parent}) + 1$$

2. Relationship update:

$$C(b_{parent}) := C(b_{parent}) \cup \{b_{new}\}$$

3. Tip update:

$$T := (T \setminus \{b_{parent}\}) \cup \{b_{new}\}$$

4. Best height update:

$$\textit{If } r(b_{new}) > h_{best}, \textit{ then } h_{best} := r(b_{new})$$

Following insertion, the tree is automatically pruned in two phases:

1. **Backward pruning**: Remove orphaned forks from tips as defined in section 9.2.4, enabling forward progress
2. **Forward pruning**: Remove finalized blocks from the elder as defined in section 9.2.5, advancing the elder forward

This two-phase approach ensures that orphaned forks don't block forward pruning of the canonical chain, that unresolved forks prevent premature finalization, and that the elder always represents the oldest block in any active chain.

9.2.3 Pruning Criteria

A block b becomes eligible for pruning when:

$$r(b) \leq h_{best} - (d_{conf} - 1) \tag{20}$$

This defines the **pruning threshold**:

$$h_{prune} = h_{best} - (d_{conf} - 1) \tag{21}$$

9.2.4 Backward Pruning (Orphaned)

For each tip $t \in T$, the algorithm checks if the tip represents an orphaned fork by traversing backward toward the elder. A tip t is orphaned if:

$$r(t) \leq h_{prune} \tag{22}$$

The backward traversal removes parent-child relationships and prunes childless blocks:

1. For a block b with child c (or tip if $c = \emptyset$):
   - *If* $r(b) \leq h_{prune}$ and a child exists:
   $$C(b) := C(b) \setminus \{c\}$$
   - *If* $C(b) = \emptyset$ after removal: remove b from B and classify as **orphaned**
   - Recursively process parent $p(b)$
2. Termination occurs when:
   - $r(b) > h_{prune}$ (block is too recent), or
   - The parent has been reached that still has other children: $C(b) \neq \emptyset$

9.2.5 Forward Pruning (Finalized)

Starting from the elder e, the algorithm prunes finalized blocks that satisfy all of the following conditions:

1. Depth condition:
   $$r(e) \leq h_{prune}$$
2. No fork condition (exactly one child):
   $$|C(e)| = 1$$

The forward pruning process recursively follows the chain:

$$e' = \begin{cases} c & \text{if } r(e) \leq h_{prune} \wedge |C(e)| = 1 \\ e & \text{otherwise} \end{cases} \tag{23}$$

where c is the unique child in $C(e)$. Each pruned block is classified as **finalized**.

The algorithm terminates when it encounters a block that either:

- Is above the pruning threshold: $r(e) > h_{prune}$, or
- Has multiple children: $|C(e)| > 1$, indicating an unresolved fork

9.2.6 Block Classification

Pruned blocks are classified as:

$$\text{fate}(b) = \begin{cases} \text{Finalized} & \text{if pruned via forward traversal} \\ \text{Orphaned} & \text{if pruned via backward traversal} \end{cases} \tag{24}$$

This classification determines the handling of associated pegouts: finalized blocks indicate permanent confirmation, while orphaned blocks require returning pegouts to the pending set.